

Become a Full Stack developer - Exercises

The goal of these exercises is to get a clear knowledge of how web applications work in its whole, with independence of the technology, frameworks, or hosting used.

You will develop a ToDo list application where you will be able to:

1. READ, INSERT, EDIT, DELETE ToDo items in/from a DB table via SQL statements
2. Develop a server that exposes API endpoints for READ, INSERT, EDIT, DELETE the ToDo items.
3. Do HTTP Requests to those endpoints using the browser && Postman
4. Inspect the HTTP trace, to understand what's happening behind the scenes
5. Create a Frontend application that first runs independently with static data, and later gets data from DB using the API endpoints

Part 1: Understand how a database works

Here you have to create a database instance of your choice. There is no need to deploy it in any cloud environment, just create a DB instance in your computer, and learn how to execute SQL statements against it.

Follow this tutorial: <https://www.youtube.com/watch?v=QjICgmk31js>

EXERCISES:

1. Save your work (.sql files, .csv files, .txt files, etc...). Save whatever you generate in a specific folder "Exercise 1". So later you can review it with a colleague, ask answer questions, or let your colleagues review your work and give you tips.

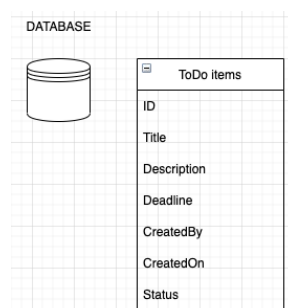
Copy those files in a shared folder of a git repo and paste the link here:

2. Save screenshots of your statements in the console. Paste those screenshots in this document. Put here all the screenshots you want, choose those which you think are more interesting. It will show your work, but also will help you to remember how did you do certain things. Feel free to be creative and go one step further, executing extra queries that don't appear in the tutorial, but help you to understand more about SQL.

SCREENSHOTS:

3. Create the ToDo Items table

Now you have to create the table you will use in your app. Just create a table with the following columns



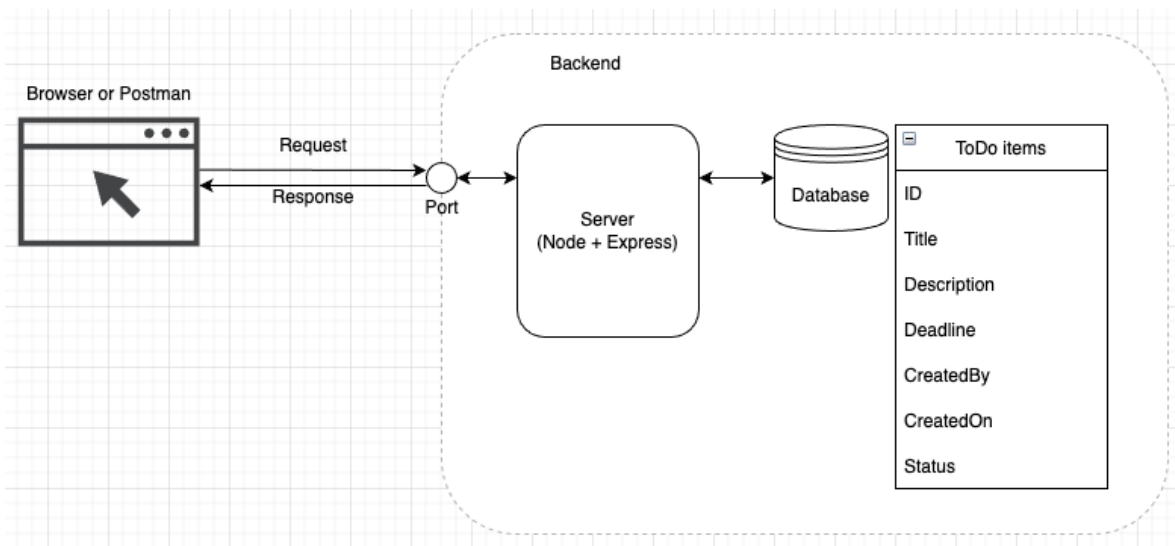
Part 2: Create your server

Your database is ready, but now you can only access it using the console on your computer. We want to be able to access its data using the Web Browser. This means, we want to access it using the HTTP protocol. To do that we need to create a Server.

The server is just an application (a bunch of code) running in your computer, that listens in a specific in/out port. Whenever you send a request to that port, that application will attend it and do something. If the request met some criteria, it will be accepted and generate a response (for example, a response with the data from the DB), but if the request does not meet the criteria, it will be rejected with an Error (for example, if the endpoint is not correct)

Follow this tutorial carefully: <https://www.youtube.com/watch?v=cr3pX6fSUpc>

Take your time, watch it several times if needed, but understand clearly all the steps, till the point you are able to explain yourself all the different parts of your server.



EXERCISES:

1. Save your work in a Git repository of your choice. Create a README file in your repo that explains step by step how to download and run your code.

Paste the link to your repo here:

2. Transform the tutorial code or create a new project, so you can CREATE, READ, UPDATE, DELETE ToDo items instead of Cars.
Save this new code in a new branch of your repo.

Branch name:

3. Use the browser to do some GET requests against your server. Use the browser “inspect” tools (Chrome is the best choice) to inspect the network trace and see those requests and its response. Paste some screenshots here showing those requests in the network trace. Write some brief description about the most important information you can see in those requests. Make sure you do as well some requests that returns an error, so you can understand errors syntax too.

SCREENSHOTS:

4. Do the same as in 3 but using Postman instead of the browser, and do some POST, PUT, PATCH, DELETE, requests. Then save some screenshots as well.

SCREENSHOTS:

5. Questions:
 - a. In which port is running your server? 1337
 - b. Which browser URL sends requests to your server? `http://localhost:1337/ToDoItems`
 - c. Which endpoint returns data about your ToDo items?
 - d. Write the fully qualified URL you would use to get all the ToDo items in your database
 - e. Write the fully qualified URL you would use to get one specific ToDo item in your database
 - f. Explain in detail how you would insert a new item in your DB
 - g. Explain in detail how you would edit an specific item of your DB
 - h. Explain in detail how you would delete an item of your DB: For f,g,h easy with postman, editing or deleting an item requires `/<ID>` at the end.

Part 3: Create your Client application (Frontend)

The side that send calls to the server we call it "Client". Like in a restaurant, the Client requests, the server (the waiter) serves (give responses).

Up to now you have query data using the browser inspect tool or postman, but that data is not easy to see for a "non-engineer" user. Your goal now is to present that data in a nice way.

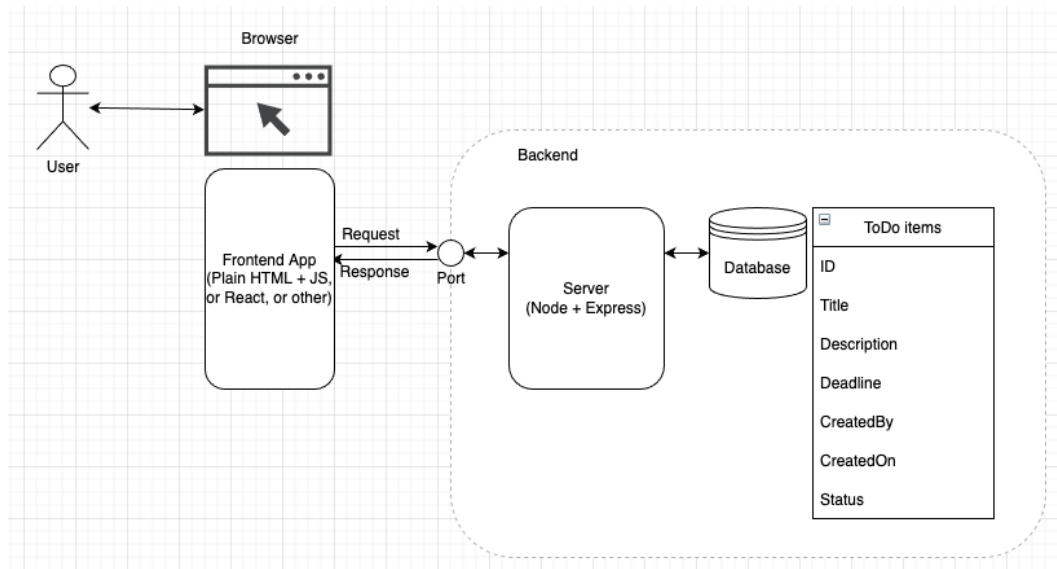
For that you have to create an independent frontend application. If you feel strong you can choose the framework of your choice, if not, my recommendation would be to use just plain HTML + Fetch or React.

I recommend you to build your frontend app using static data that you can write in a JSON file or just a JSON object somewhere in your code first, and once you have done that, you can change the parts of your code where you retrieve data from the JSON file and do some requests to your server. Remember, for that to work, you should have your Server running in port XXXX and at the same time, your Frontend app running in your browser (if you are using react, you will have your FE app running in another port, different than the server)

For HTML and Fetch (30 mins): <https://www.youtube.com/watch?v=Oive66jrwBs>

For React, watch this React crash course (1h30): <https://www.youtube.com/watch?v=w7ejDZ8SWv8>

- This is a very complete React crash course, it has more things that what you need, but is good to learn. This channel "[Traversy Media](#)" is AMAZING. Really recommend you to follow it.



EXERCISES:

1. Save your code in a GIT repo, including a README file explaining how to execute and use your frontend app.

GIT Repo URL:

2. Paste here some screenshots of your app.